

CS 504 Project Stage 3

Jonathan Dale

2026-04-12

Project stage 3

Runnable DDL for both features lives in Stage_3.sql and should be applied **after** load_data.sql so base tables (including `member_tbl`) already exist.

Feature 1.

Overdue Alert

In this feature, we are tasked with creating a mechanism to send automatic alerts to library staff when a material is overdue on a daily basis. The explanation includes:

- How overdue materials are identified
- When the alert is generated
- Any necessary updates or additions to tables, views, or schemas

Solution Material becomes past due when current date is after the `due_date` (from the `borrow` table) and `return_date` is NULL, meaning it has not been returned yet.

For this feature I will build a view to query over the live state of the data without storing redundant information. This becomes the single source of truth for any material that is currently overdue.

```
-- Create a view to identify overdue materials
CREATE OR REPLACE VIEW overdue_materials AS
SELECT
```

```
    b.borrow_id,
    b.material_id,
    m.title,
    b.member_id,
    mem.name      AS member_name,
    mem.contact_info AS member_contact,
    b.staff_id,
    b.due_date,
```

```

        CURRENT_DATE - b.due_date AS days_overdue
FROM borrow b
JOIN material m ON b.material_id = m.material_id
JOIN member_tbl mem ON b.member_id = mem.member_id
WHERE b.return_date IS NULL
      AND b.due_date < CURRENT_DATE;

```

Schema modifications

Add the overdue_alert_log table I will add a table to the database that logs when an overdue alert is generated and how it was sent. The `UNIQUE (borrow_id, alerted_on)` constraint prevents more than one alert row for the same borrow on the same calendar day.

```

-- Log when staff are alerted about an overdue borrow (at most once per borrow per day)
CREATE TABLE overdue_alert_log (
    alert_id      SERIAL PRIMARY KEY,
    borrow_id     INTEGER REFERENCES borrow(borrow_id),
    staff_id      INTEGER REFERENCES staff(staff_id),
    alerted_on    DATE NOT NULL DEFAULT CURRENT_DATE,
    alert_method  TEXT DEFAULT 'email',
    UNIQUE (borrow_id, alerted_on)
);

```

Triggers and scheduling

PostgreSQL does not have a built-in job scheduler; the `pg_cron` extension is commonly available on hosted Postgres to schedule SQL.

A row trigger only fires on an insert, update, or delete. Library materials become overdue as **calendar time passes** relative to an existing `due_date`, so a **scheduled batch** (below) is what actually runs every day. Optional row-level triggers can still be used for other rules; here the daily function is the main driver for alerts.

Data integrity (borrow)

Reasonable checks on `borrow` include: `due_date` should be on or after `borrow_date` at checkout, and when a row represents an open loan, `return_date` is NULL; when the item is returned, `return_date` should be set (typically to a date on or after `borrow_date`). These are business rules you can enforce with `CHECK` constraints or application validation; they are separate from “overdue,” which is defined against `CURRENT_DATE`.

```

-- Daily batch: log staff alerts and record overdue occurrences (Feature 2 uses the latter)
CREATE OR REPLACE FUNCTION run_daily_overdue_alerts()
RETURNS INTEGER
LANGUAGE plpgsql
AS $$

```

```

DECLARE
    inserted_alerts INTEGER;
BEGIN
    INSERT INTO overdue_alert_log (borrow_id, staff_id, alert_method)
    SELECT om.borrow_id, om.staff_id, 'email'
    FROM overdue_materials om
    WHERE NOT EXISTS (
        SELECT 1
        FROM overdue_alert_log al
        WHERE al.borrow_id = om.borrow_id
        AND al.alerted_on = CURRENT_DATE
    );

    GET DIAGNOSTICS inserted_alerts = ROW_COUNT;

    INSERT INTO overdue_occurrences (member_id, borrow_id, flagged_on)
    SELECT b.member_id, b.borrow_id, CURRENT_DATE
    FROM borrow b
    WHERE b.return_date IS NULL
        AND b.due_date < CURRENT_DATE
        AND NOT EXISTS (
            SELECT 1
            FROM overdue_occurrences oc
            WHERE oc.borrow_id = b.borrow_id
        );

    RETURN inserted_alerts;
END;
$$;

```

The function returns the number of **new alert log rows** inserted that run; overdue occurrence rows are inserted in the same transaction (see Feature 2).

Triggering the alert

Case 1. pg_cron is available Requires the extension (superuser or allowed role), for example `CREATE EXTENSION IF NOT EXISTS pg_cron;`, and a platform that supports it.

Schedule every day at 7:00 a.m.:

```
SELECT cron.schedule('daily-overdue-alert', '0 7 * * *', 'SELECT run_daily_overdue_alerts());
```

(Case 1 uses 7:00 a.m.; Case 2 below uses 00:05—either schedule is valid as long as the job runs once per day.)

Case 2. Crontab If `pg_cron` is not available, use the system crontab to run `psql` once per day (for example five minutes after midnight):

```
5 0 * * * PGDATABASE=cs_504 PGUSER=postgres /full/path/to/CS504/cron/run_overdue_alerts.sh >
```

The wrapper script `cron/run_overdue_alerts.sh` runs `psql` with `ON_ERROR_STOP` and executes `cron/overdue_alerts_daily.sql`:

```
set -eu
cd "$(dirname "$0")"
exec psql -v ON_ERROR_STOP=1 -f overdue_alerts_daily.sql
overdue_alerts_daily.sql:
SELECT run_daily_overdue_alerts() AS rows_inserted;
```

Feature 2.

Membership deactivation and reactivation based on overdue behavior

I will add a `status` column on `member_tbl` indicating whether membership is 'active' or 'inactive', using `ALTER TABLE`.

```
ALTER TABLE member_tbl
    ADD COLUMN status TEXT NOT NULL DEFAULT 'active'
    CHECK (status IN ('active', 'inactive'));
```

The `CHECK` constraint prevents values other than `active` or `inactive`.

Definition of an overdue occurrence

For this design, **one overdue occurrence means one distinct loan (`borrow_id`) that has become overdue at least once**: the first calendar day the batch finds that loan with `return_date` IS NULL and `due_date` < `CURRENT_DATE`, it inserts one row into `overdue_occurrences` for that `borrow_id`. The `UNIQUE(borrow_id)` constraint ensures the same loan is not counted multiple times on later days. This is different from counting each overdue day separately or each daily staff alert.

Next I add a table that stores those occurrences per member; after **three or more** such rows for a member, the member is marked `inactive`.

```
CREATE TABLE overdue_occurrences (
    occurrence_id SERIAL PRIMARY KEY,
    member_id     INTEGER REFERENCES member_tbl(member_id),
    borrow_id     INTEGER REFERENCES borrow(borrow_id) UNIQUE,
    flagged_on    DATE NOT NULL DEFAULT CURRENT_DATE
);
```

Occurrence rows are inserted inside `run_daily_overdue_alerts()` (same scheduled job as Feature 1), so tracking stays aligned with the daily overdue check.

After that, a trigger on `overdue_occurrences` deactivates the member when they reach three or more occurrences.

```

-- deactivate member if 3 or more overdue occurrences exist
CREATE OR REPLACE FUNCTION check_and_deactivate_member()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    occurrence_count INTEGER;
BEGIN
    SELECT COUNT(*) INTO occurrence_count
    FROM overdue_occurrences
    WHERE member_id = NEW.member_id;

    IF occurrence_count >= 3 THEN
        UPDATE member_tbl
        SET status = 'inactive'
        WHERE member_id = NEW.member_id
        AND status = 'active';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_deactivate_member
AFTER INSERT ON overdue_occurrences
FOR EACH ROW
EXECUTE FUNCTION check_and_deactivate_member();

```

For membership **reactivation**, I add a table to record overdue fee payments.

```

-- create overdue fee payment table
CREATE TABLE fee_payment (
    payment_id SERIAL PRIMARY KEY,
    member_id INTEGER REFERENCES member_tbl(member_id),
    paid_on DATE NOT NULL DEFAULT CURRENT_DATE,
    amount NUMERIC(8, 2) NOT NULL CHECK (amount > 0),
    staff_id INTEGER REFERENCES staff(staff_id),
    payment_notes TEXT
);

```

amount must be greater than zero. **Simplification:** the system does not compute a balance from fines or tie the payment to specific borrows; it treats a positive `fee_payment` row with `paid_on` date on or after the member's latest `flagged_on` date in `overdue_occurrences` as sufficient evidence that the overdue fee was paid.

For automatic reactivation, a trigger runs after each insert to `fee_payment`. It sets the member to `active` only when they were `inactive`, and clears `overdue_occurrences` for that member **only if** that update actually ran (so arbitrary payments while already active do not wipe history).

```

-- reactivate after overdue fee payment is received
CREATE OR REPLACE FUNCTION check_and_reactivate_member()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
DECLARE
    latest_occurrence DATE;
BEGIN
    SELECT MAX(flagged_on) INTO latest_occurrence
    FROM overdue_occurrences
    WHERE member_id = NEW.member_id;

    IF latest_occurrence IS NOT NULL AND NEW.paid_on >= latest_occurrence THEN
        UPDATE member_tbl
        SET status = 'active'
        WHERE member_id = NEW.member_id
        AND status = 'inactive';

        IF FOUND THEN
            DELETE FROM overdue_occurrences
            WHERE member_id = NEW.member_id;
        END IF;
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_reactivate_member
AFTER INSERT ON fee_payment
FOR EACH ROW
EXECUTE FUNCTION check_and_reactivate_member();

```

After an inactive member pays the overdue fee and is reactivated, their occurrence rows are removed from `overdue_occurrences` so the counter starts fresh at zero.